E2. Integrative Project: Streaming Service Modeling

Object-Oriented Programming (Gpo 304)

Omar Del Toro Covarrubias - A01648573

Gabriel Vallarta Ramírez -A01648413

Professor Carlos Ramón Garibay Guémez

Tecnológico de Monterrey - Campus Guadalajara

June 10th, 2026

Table of Contents

1. Introduction

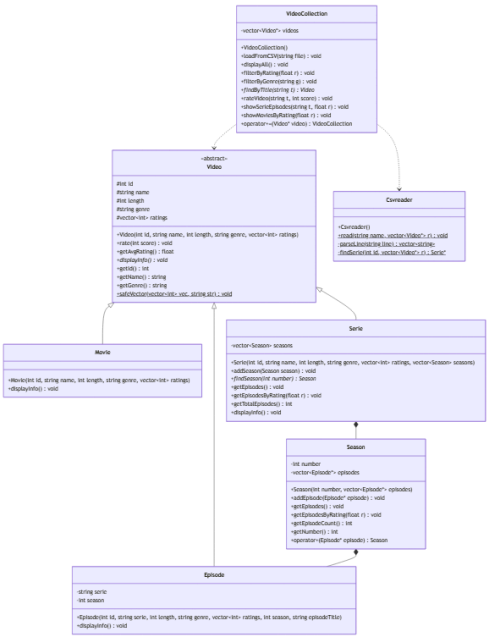
In recent years, low cost, on-demand streaming services such as Netflix, Disney+, and HBO Max have become increasingly popular. Some of these services are focused on the volume of videos made available to users, while others are focused on showcasing exclusive their own branded content.

This project represents the design and implementation of a limited version of a content provider system intended to support these types of streaming services. The system handles two kinds of videos: movies and series. Every video has an ID, a name, a length, and a genre (for example, Drama, Action, Mystery). Series are further organized into seasons, each containing a set of episodes. Every episode has a title and belongs to a specific season.

The system also supports a rating mechanism, where each video and episode can receive ratings on a scale from 1 to 5, and the average rating is computed and displayed. The application provides a menu-driven interface that allows the user to filter videos by rating or genre, view episodes of a specific series, rate a video, and more.

2. UML Class Diagram

The following UML class diagram represents the full object-oriented design of the system, including all classes, their attributes, methods, and relationships:



For a more detailed view, the complete class diagram is available in this link

2.1 Design Justification

The design is structured around a class hierarchy rooted in an abstract base class Video, which captures the common attributes and behaviors shared by all video types.

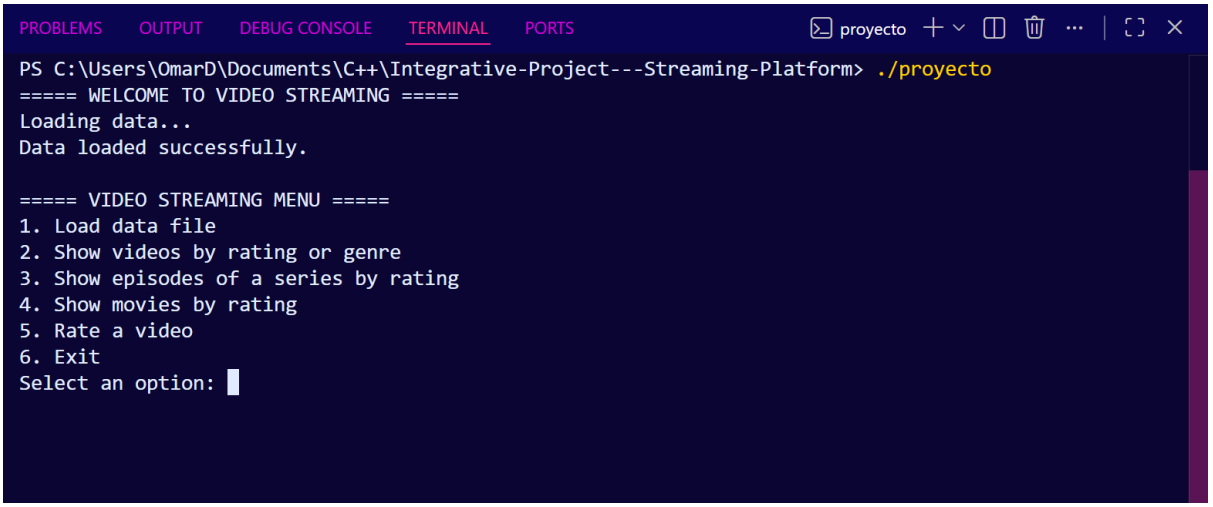
- **Video (abstract)**: Defines the common interface and shared attributes, such as id, name, length, genre, and ratings for all video types. The displayInfo() method is declared as a pure virtual function, enforcing that every concrete subclass provides its own implementation. This ensures polymorphic behavior throughout the system.
- **Movie**: Inherits directly from Video. Its only responsibility is to override displayInfo() to present movie-specific formatting. All other behavior is inherited.
- **Serie**: Also inherits from Video and adds a collection of Season objects. It overrides displayInfo() and provides additional methods to navigate its episodes and seasons, such as getEpisodes(), getEpisodesByRating(), findSeason(), and getTotalEpisodes().
- **Episode**: Inherits from Video as well. This decision was made because an episode shares the same fundamental properties as any other video: it has a name (episode

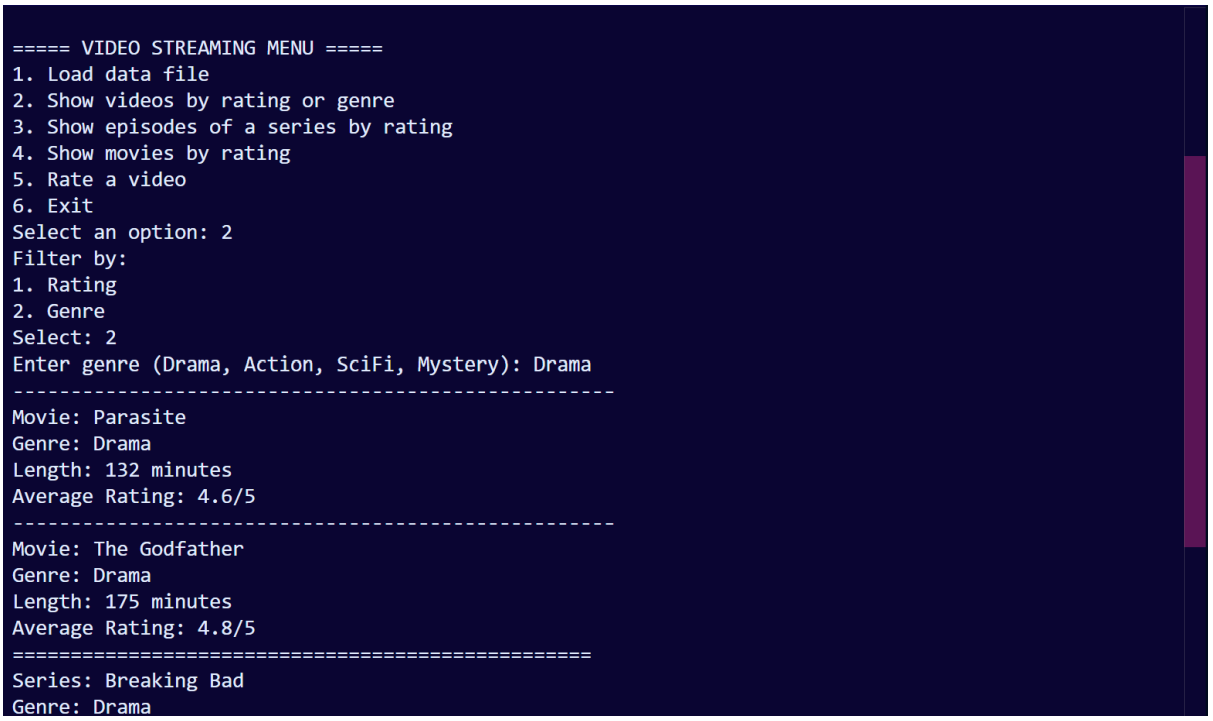
title), a length, a genre, and its own set of ratings. The series name and season number are stored as additional attributes to provide context when displaying episode information.

- **Season**: A non-Video class that acts as an intermediary container between Serie and Episode. It holds a collection of Episode pointers and provides methods to display and filter episodes. The operator+ is overloaded in Season to allow adding an episode using the + operator, demonstrating operator overloading in a natural context.
- **VideoCollection**: Serves as the main controller of the system. It holds a vector of Video pointers, enabling polymorphic storage of both Movie and Serie objects. It provides all the methods required by the application menu, including filtering, searching, and rating. The operator+= is overloaded to allow adding videos to the collection in an expressive way.
- **CSVReader**: A utility class responsible for reading and parsing the CSV data file. It is completely decoupled from the domain classes, following the single responsibility principle. Its methods are static since they do not depend on any instance state.

3. Execution Example

The following screenshots show the system running. The program loads the data automatically on startup and presents the user with a menu to interact with the video collection.





```
===== VIDEO STREAMING MENU =====
1. Load data file
2. Show videos by rating or genre
3. Show episodes of a series by rating
4. Show movies by rating
5. Rate a video
6. Exit
Select an option: 3
Enter series title: Breaking Bad
Enter minimum episode rating (1.0 - 5.0): 3
Series: Breaking Bad
Season 1:
--------------------------------------------------
Serie: Breaking Bad, season 1
Episode title: Pilot
Genre: Drama
Length: 45 minutes
Average Rating: 4.66667/5
---
--------------------------------------------------
Serie: Breaking Bad, season 1
Episode title: Cat's in the Bag
Genre: Drama
Length: 45 minutes
Average Rating: 3.66667/5
```

```
===== VIDEO STREAMING MENU =====
1. Load data file
2. Show videos by rating or genre
3. Show episodes of a series by rating
4. Show movies by rating
5. Rate a video
6. Exit
Select an option: 5
Enter video title: Parasite
Enter rating (1 - 5): 1
Rating added successfully.
```

4. Argument: OOP Requirements

The following section justifies how the implementation satisfies each of the evaluated requirements:

a) Proper classes are identified

Seven classes were identified and implemented: Video, Movie, Serie, Episode, Season, VideoCollection, and CSVReader. Each class has a well-defined responsibility aligned with the problem domain. The separation between domain classes (Video hierarchy) and support classes (VideoCollection, CSVReader) reflects a clean design.

b) Inheritance is implemented properly

Inheritance is used in three relationships: Movie, Serie, and Episode all inherit from the abstract class Video. This allows shared attributes and behaviors (id, name, length, genre, ratings, rate(), getAvgRating()) to be defined once in the base class and reused across all subclasses, avoiding code duplication.

c) Access modifiers are implemented properly

The protected modifier is used in Video for attributes that need to be accessible in subclasses (id, name, length, genre, ratings). All class-specific attributes in subclasses (for example, seasons in Serie, serie and season in Episode) are declared private. Public methods provide the necessary interface for external interaction.

d) Method overriding is implemented properly

The displayInfo() method is declared as a pure virtual function in Video and is overridden in Movie, Serie, and Episode. Each override provides a different output format appropriate to the video type. The override keyword is used explicitly in all subclasses to enforce correctness at compile time.

e) Polymorphism is implemented properly

Polymorphism is achieved through the use of Video* pointers in VideoCollection. Both Movie and Serie objects are stored in the same vector<Video*> and their displayInfo() method is called polymorphically. Additionally, dynamic_cast is used in VideoCollection to safely downcast Video* pointers to Serie* or Movie* when type-specific operations are needed.

f) Abstract classes are implemented properly

Video is declared as an abstract class by defining displayInfo() as a pure virtual function, which means it equals zero. This prevents direct instantiation of Video and enforces that every concrete subclass provides its own implementation. The system correctly uses Video only through pointers, never instantiating it directly.

g) At least one operator is overloaded properly

Two operators are overloaded in the implementation. The operator+ is overloaded in Season to add an Episode* to the season's episode list, returning a Season& to allow chaining. The operator+= is overloaded in VideoCollection to add a Video* to the collection, also returning a reference to the collection. Both are semantically meaningful and consistent with standard C++ conventions.

h) Exception handling

Basic defensive programming is implemented throughout the system: the CSV reader checks whether the file opened successfully before reading, all menu inputs are validated with range and type checks, and methods that search by title return nullptr and display an informative message when no match is found. Formal try/catch exception handling was not implemented in this project

5. Cases That Would Prevent the Project from Functioning Properly

The following cases were identified as potential failure points in the current implementation:

- **Missing or misnamed CSV file**: If videos.csv is not present in the same directory as the executable, the program will display an error message and the video collection will remain empty. All menu options will return no results.
- **Malformed CSV structure**: If the CSV file has missing columns, incorrect column order, or inconsistent formatting, the parser may crash or produce incorrect data. For example, a missing season number field for a series row would cause stoi() to fail.
- **Ratings greater than single digits**: The safeVector() function extracts individual digits from the ratings string. If a rating value were ever two digits (for example, 10), it would be read as two separate values (1 and 0) instead of one, producing incorrect averages.
- **Case-sensitive genre and title search**: The filterByGenre() and findByTitle() methods perform exact string comparisons. If the user enters 'drama' instead of 'Drama', no results will be found. There is no case-insensitive normalization.
- **Loading the CSV multiple times**: If the user selects option 1 from the menu more than once, the videos vector will accumulate duplicates, leading to repeated entries in all filter and display operations.
- **Memory management**: The system uses raw pointers (new Movie, new Serie, new Episode) without explicit delete calls. While this is acceptable for a short-lived program, it constitutes a memory leak and would be a problem in a long-running or production system.

6. Personal Conclusions

This project provided a practical opportunity to apply Object-Oriented Programming concepts in a real-world modeling scenario. Designing and implementing the class hierarchy for a streaming service system reinforced the importance of abstraction, encapsulation, and the proper use of inheritance and polymorphism.

One of the most valuable lessons was understanding when and why to use abstract classes. Declaring Video as abstract with a pure virtual displayInfo() method ensured that every concrete video type provides its own output, while still allowing the system to treat all videos uniformly through pointers. This is the essence of polymorphism in practice.

The decision to make Episode inherit from Video, rather than being a standalone class, made sense because an episode shares the same fundamental properties as any other video. This decision simplified the class hierarchy and allowed the rating system to be reused without duplication.

The implementation of operator overloading in Season and VideoCollection made the code more expressive and demonstrated how C++ operators can be extended within a domain context.

Overall, this project strengthened our understanding of OOP design principles and their translation into working C++ code, from initial UML modeling through to a functional menu-driven application.

7. References

Source files included in this submission:

- video.h / video.cpp — Abstract base class
- movie.h / movie.cpp — Movie class
- serie.h / serie.cpp — Serie class
- season.h / season.cpp — Season class
- episode.h / episode.cpp — Episode class
- videoCollection.h / videoCollection.cpp — Collection and menu logic
- csvreader.h / csvreader.cpp — CSV file parser
- main.cpp — Application entry point and menu
- videos.csv — Data file with movies and series
- class_diagram.png — UML class diagram with all the classes and relationships between them

To compile and run the project:

```
g++ *.cpp -o proyecto.exe
```

```
./proyecto
```

Make sure videos.csv is in the same directory as the compiled executable.